

Predicting Heart Disease using Machine Learning Algorithms

MAC Mathematical Association Of Cambodia

Trainer: Kimsie PHAN

Prepared By

Rachanyt HEN & Pheany PHAL



November 29, 2025

Content

- 1 Introduction
- 2 Dataset Description
- 3 Preprocessing and cleaning dataset
- 4 Distribution Dataset
 - Research questions
- 5 Machine Learning Algorithms
 - Conclusion
 - Future Work

Content

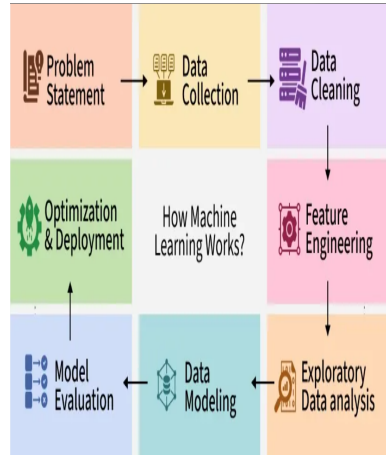
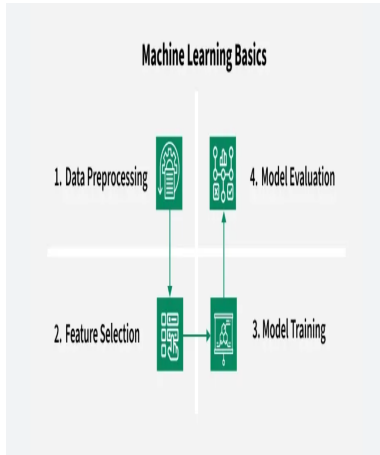
- 1 Introduction
- 2 Dataset Description
- 3 Preprocessing and cleaning dataset
- 4 Distribution Dataset
 - Research questions
- 5 Machine Learning Algorithms
 - Conclusion
 - Future Work

Introduction

Heart disease is the world's most critical health issue, affecting millions of lives each years. But what if we could predict it earlier using technology?

In this presentation, we will explore how to use a real world patient dataset including factors like age, cholesterol levels, chest pain type etc. to predict heart disease. You will see how to do preprocessing and cleaning the dataset for models, analyze the relationships between different health factors, and train multiple machine learning models to identify the most accurate predictor.

We'll conclude by sharing our final results and look forward to answering your questions.



Content

- 1 Introduction
- 2 Dataset Description
- 3 Preprocessing and cleaning dataset
- 4 Distribution Dataset
 - Research questions
- 5 Machine Learning Algorithms
 - Conclusion
 - Future Work

Dataset Description

```
import pandas as pd  
df = pd.read_csv("heart_disease.csv")  
df
```

	id	age	sex	dataset	cp	trestbps	chol	fbs	restecg	thalch	exang	oldpeak	slope	ca	thal	num
0	1	63	Male	Cleveland	typical angina	145.0	233.0	True	lv hypertrophy	150.0	False	2.3	downsloping	0.0	fixed defect	0
1	2	67	Male	Cleveland	asymptomatic	160.0	286.0	False	lv hypertrophy	108.0	True	1.5	flat	3.0	normal	2
2	3	67	Male	Cleveland	asymptomatic	120.0	229.0	False	lv hypertrophy	129.0	True	2.6	flat	2.0	reversible defect	1
3	4	37	Male	Cleveland	non-anginal	130.0	250.0	False	normal	187.0	False	3.5	downsloping	0.0	normal	0
4	5	41	Female	Cleveland	atypical angina	130.0	204.0	False	lv hypertrophy	172.0	False	1.4	upsloping	0.0	normal	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
915	916	54	Female	VA Long Beach	asymptomatic	127.0	333.0	True	st-t abnormality	154.0	False	0.0	NaN	NaN	NaN	1
916	917	62	Male	VA Long Beach	typical angina	NaN	139.0	False	st-t abnormality	NaN	NaN	NaN	NaN	NaN	NaN	0
917	918	55	Male	VA Long Beach	asymptomatic	122.0	223.0	True	st-t abnormality	100.0	False	0.0	NaN	NaN	fixed defect	2
918	919	58	Male	VA Long Beach	asymptomatic	NaN	385.0	True	lv hypertrophy	NaN	NaN	NaN	NaN	NaN	NaN	0
919	920	62	Male	VA Long Beach	atypical angina	120.0	254.0	False	lv hypertrophy	93.0	True	0.0	NaN	NaN	NaN	1

920 rows × 16 columns

- *age* : The individual's age in years
- *sex* : The individual's. Sex(taking male=1 and female =0)
- *dataset* : Where the record from (Cleveland,Hungary, Switzerland, VA Long Beach)
- *cp* : The chest pain experienced(1: typical angina, 2: atypical angina, 3: non-anginal pain,4: asymptomatic)
- *trestbps*: The person's resting blood pressure (mm Hg on admission to the hospital)
- *chol* : The person's cholesterol measurement in *mg/dl*
- *fbs* : The person's fasting blood sugar
($> 120\text{mg/dl}$, $1 = \text{true}$; $0 = \text{false}$)
- *restecg* : Resting electrocardiographic measurement (0 = normal, 1 = having ST-T wave abnormality, 2 = showing probable or definite left ventricular hypertrophy by Estes' criteria)

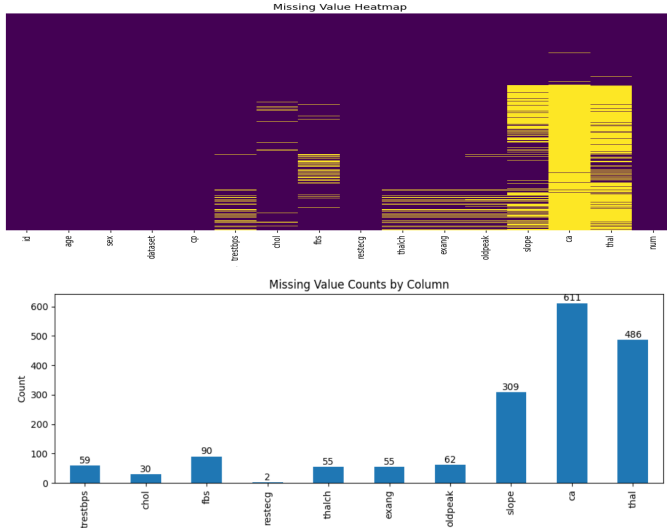
Dataset Description

- *thalach* : The person's maximum heart rate achieved
- *exang* : Exercise induced angina (1 = yes; 0 = no)
- *oldpeak* : ST depression induced by exercise relative to rest ('ST' relates to positions on the ECG plot.)
- *slope* : the slope of the peak exercise ST segment (1: upsloping, 2: flat, 3: downsloping)
- *ca* : The number of major vessels (0-3)
- *thal* : A blood disorder called thalassemia (3 = normal; 6 = fixed defect; 7 = reversable defect)
- *num or target* : Heart Disease (0=no, greater than 0 =yes)

Content

- 1 Introduction
- 2 Dataset Description
- 3 Preprocessing and cleaning dataset
- 4 Distribution Dataset
 - Research questions
- 5 Machine Learning Algorithms
 - Conclusion
 - Future Work

Preprocessing and cleaning Dataset



Handle missing values for visualization purposes

```
df = df.drop(columns=['slope', 'ca', 'thal'])
df = df.dropna()
```

We have removed 180 *rows* and 3 *columns* with missing values. So our dataset is in size 740*columns* × 13*rows*.

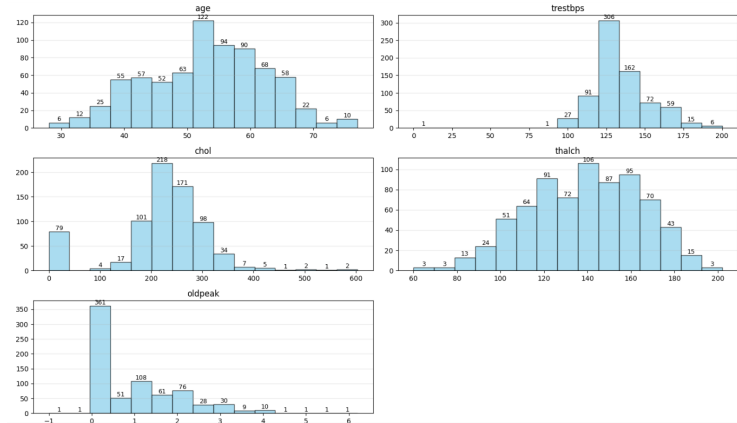
	id	age	sex	dataset		cp	trestbps	chol	fbs	restecg	thalch	exang	oldpeak	target
	0	1	63	Male	Cleveland	typical angina	145.0	233.0	True	lv hypertrophy	150.0	False	2.3	0
	1	2	67	Male	Cleveland	asymptomatic	160.0	286.0	False	lv hypertrophy	108.0	True	1.5	1
	2	3	67	Male	Cleveland	asymptomatic	120.0	229.0	False	lv hypertrophy	129.0	True	2.6	1
	3	4	37	Male	Cleveland	non-anginal	130.0	250.0	False	normal	187.0	False	3.5	0
	4	5	41	Female	Cleveland	atypical angina	130.0	204.0	False	lv hypertrophy	172.0	False	1.4	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
913	914	62	Male	VA Long Beach	asymptomatic	158.0	170.0	False	st-t abnormality	138.0	True	0.0	1	
914	915	46	Male	VA Long Beach	asymptomatic	134.0	310.0	False	normal	126.0	False	0.0	1	
915	916	54	Female	VA Long Beach	asymptomatic	127.0	333.0	True	st-t abnormality	154.0	False	0.0	1	
917	918	55	Male	VA Long Beach	asymptomatic	122.0	223.0	True	st-t abnormality	100.0	False	0.0	1	
919	920	62	Male	VA Long Beach	atypical angina	120.0	254.0	False	lv hypertrophy	93.0	True	0.0		

740 rows × 13 columns

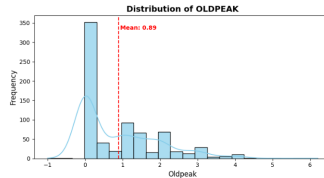
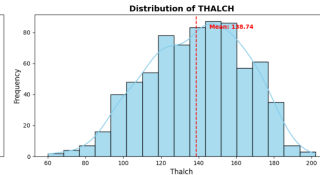
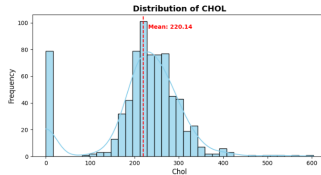
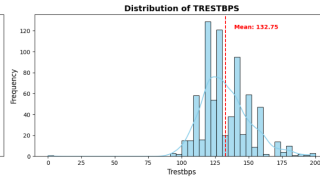
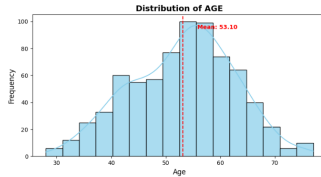
Content

- 1 Introduction
- 2 Dataset Description
- 3 Preprocessing and cleaning dataset
- 4 Distribution Dataset**
 - Research questions
- 5 Machine Learning Algorithms
 - Conclusion
 - Future Work

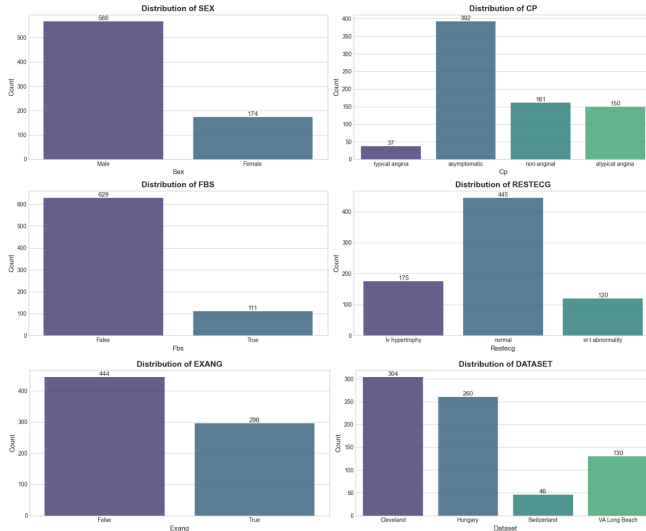
Distribution of Numerical features



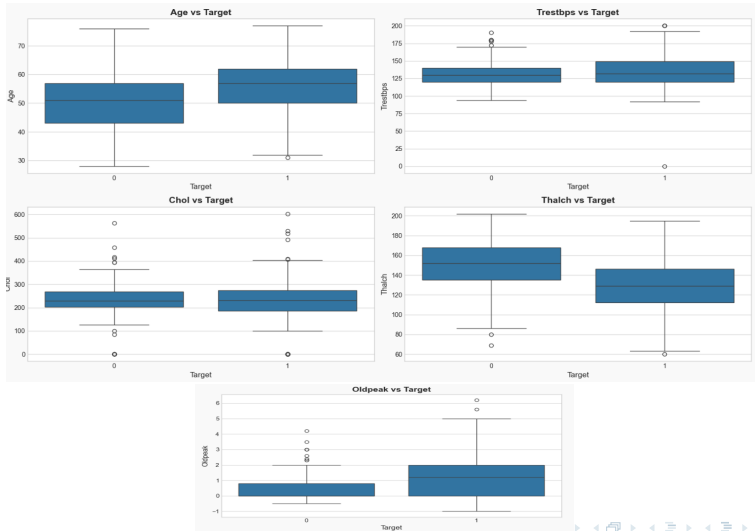
Histograms of Numerical features

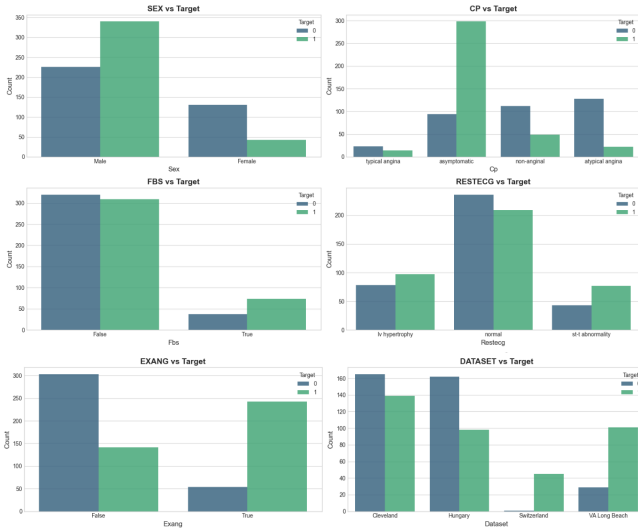


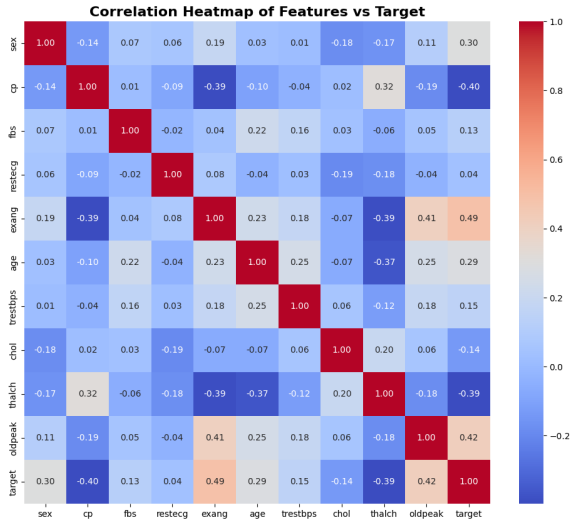
Distribution of Categorical features



Features Vs. Target

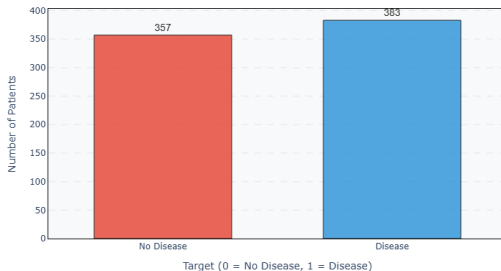






1. *How many patients have heart disease vs no disease?*

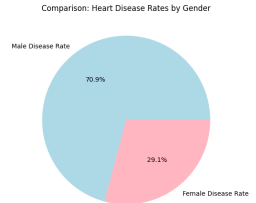
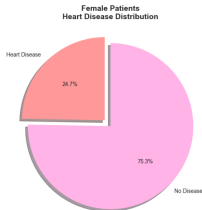
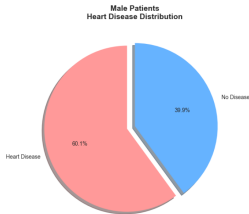
Heart Disease Distribution in Patients



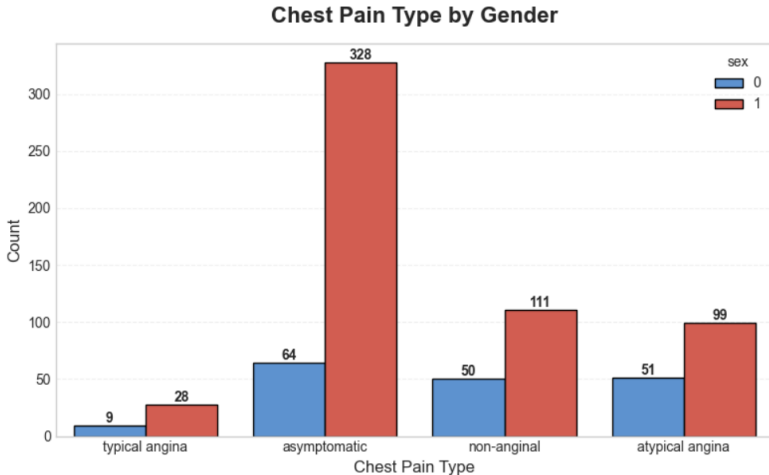
- No Disease : 357 (48.24%)
- Disease : 383 (51.76%)

2. Do males or females have a higher risk of heart disease?

- Total males: 566
- Total females: 174
- Males with heart disease: 340 (60.1%)
- Females with heart disease: 43 (24.7%)



3. Which chest pain types are most common?



Content

- 1 Introduction
- 2 Dataset Description
- 3 Preprocessing and cleaning dataset
- 4 Distribution Dataset
 - Research questions
- 5 Machine Learning Algorithms**
 - Conclusion
 - Future Work

Descriptive Statistics

	count	mean	std	min	25%	50%	75%	max
id	740.0	406.598649	259.907071	1.0	185.75	377.5	588.25	920.0
age	740.0	53.097297	9.408127	28.0	46.00	54.0	60.00	77.0
sex	740.0	0.764865	0.424370	0.0	1.00	1.0	1.00	1.0
dataset	740.0	1.002703	1.085016	0.0	0.00	1.0	1.00	3.0
cp	740.0	0.787838	0.950529	0.0	0.00	0.0	2.00	3.0
trestbps	740.0	132.754054	18.581250	0.0	120.00	130.0	140.00	200.0
chol	740.0	220.136486	93.614555	0.0	197.00	231.0	271.00	603.0
fbbs	740.0	0.150000	0.357313	0.0	0.00	0.0	0.00	1.0
restecg	740.0	0.925676	0.627421	0.0	1.00	1.0	1.00	2.0
thalch	740.0	138.744595	25.846082	60.0	120.00	140.0	159.25	202.0
exang	740.0	0.400000	0.490229	0.0	0.00	0.0	1.00	1.0
oldpeak	740.0	0.894324	1.087160	-1.0	0.00	0.5	1.50	6.2
target	740.0	0.517568	0.500029	0.0	0.00	1.0	1.00	1.0


```
# Libraries for developing algorithms
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier

# Preprocessing
numeric_features = ['age', 'trestbps', 'chol', 'thalch', 'oldpeak']
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

categorical_features = ['sex', 'cp', 'fbs', 'restecg', 'exang']
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features),
        ('cat', categorical_transformer, categorical_features)
    ])

# Define features and target variables
X = df[numeric_features + categorical_features]
y = df['target']
```

```
#Splitting Dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Define models
models = {
    'Logistic Regression': LogisticRegression(max_iter=1000),
    'Decision Tree': DecisionTreeClassifier(),
    'Random Forest': RandomForestClassifier(),
    'Support Vector Machine': SVC(probability=True),
    'K-Nearest Neighbors': KNeighborsClassifier()
}

# Evaluate models
auc_scores = {}
for model_name, model in models.items():
    clf = Pipeline(steps=[('preprocessor', preprocessor),
                          ('classifier', model)])
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)

    # Get probability estimates if possible
    if hasattr(clf, "predict_proba"):
        y_pred_proba = clf.predict_proba(X_test)[: , 1]
        auc = roc_auc_score(y_test, y_pred_proba)
        auc_scores[model_name] = auc
    else:
        # If model does not provide predict_proba, we can't calculate AUC
        auc_scores[model_name] = None

    print(f"Model: {model_name}")
    print(classification_report(y_test, y_pred))
    print("Confusion Matrix:")
    print(confusion_matrix(y_test, y_pred))
    if auc_scores[model_name] is not None:
        print(f"ROC AUC Score: {auc_scores[model_name]}")
    else:
        print("ROC AUC Score: Not available")
    print("-" * 40)

# Sort models by AUC score
sorted_auc_scores = sorted(auc_scores.items(), key=lambda x: (x[1] is not None, x[1]), reverse=True)

print("Models sorted by AUC score:")
for model_name, auc in sorted_auc_scores:
    if auc is not None:
        print(f"{model_name}: {auc:.4f}")
    else:
        print(f"{model_name}: AUC not available")
```

```
Model: Logistic Regression
precision recall f1-score support

0 0.91 0.81 0.86 75
1 0.83 0.92 0.87 73

accuracy 0.86 148
macro avg 0.87 0.87 0.86 148
weighted avg 0.87 0.86 0.86 148
```

```
Confusion Matrix:
[[61 14]
 [ 6 67]]
ROC AUC Score: 0.9136073059360731
```

```
Model: Decision Tree
precision recall f1-score support

0 0.70 0.68 0.69 75
1 0.68 0.70 0.69 73

accuracy 0.69 148
macro avg 0.69 0.69 0.69 148
weighted avg 0.69 0.69 0.69 148
```

```
Confusion Matrix:
[[51 24]
 [22 51]]
ROC AUC Score: 0.6893150684931507
```

```
Model: Random Forest
precision recall f1-score support

0 0.86 0.76 0.81 75
1 0.78 0.88 0.83 73

accuracy 0.82 148
macro avg 0.82 0.82 0.82 148
weighted avg 0.82 0.82 0.82 148
```

```
Confusion Matrix:
[[57 18]
 [ 9 64]]
ROC AUC Score: 0.8786301369863013
```

```
Model: Support Vector Machine
precision recall f1-score support

0 0.89 0.76 0.82 75
1 0.79 0.90 0.84 73

accuracy 0.83 148
macro avg 0.84 0.83 0.83 148
weighted avg 0.84 0.83 0.83 148
```

```
Confusion Matrix:
[[57 18]
 [ 7 66]]
ROC AUC Score: 0.9010045662100457
```

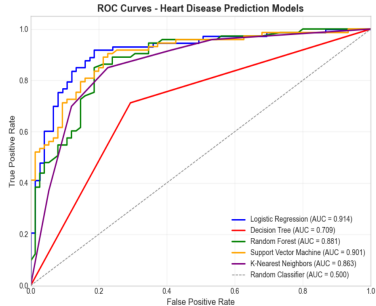
```

Model: K-Nearest Neighbors
      precision    recall  f1-score   support

      0       0.84       0.77       0.81         75
      1       0.78       0.85       0.82         73

 accuracy         0.81
 macro avg       0.81       0.81       0.81
 weighted avg    0.81       0.81       0.81

Confusion Matrix:
[[58 17]
 [11 62]]
ROC AUC Score: 0.862648401826484
-----
Models sorted by AUC score:
Logistic Regression: 0.9136
Support Vector Machine: 0.9010
Random Forest: 0.8786
K-Nearest Neighbors: 0.8626
Decision Tree: 0.6893
    
```



Final Decision Logistic Regression: Generally provides a good balance between performance and interpretability, especially for datasets with mixed types of features and potential nonlinear relationships. Its ability to provide feature importance is also valuable for understanding the impact of each attribute on the prediction.

Conclusion

Best Performer: The *Logistic Regression* model is the most effective classifier in this case, achieving the highest ROC AUC score and solid performance across all metrics.

Runner-up: The *Support Vector Machine (SVM)* is close behind, with excellent accuracy and nearly comparable ROC AUC score.

Good Alternative: *Random Forest* is also a strong contender, especially considering its simplicity and ease of interpretation. The other models, *K-Nearest Neighbors* and *Decision Tree*, did not perform well in comparison, making them less ideal for this specific dataset and problem.

Future Work

- 1 Improve missing-value handling
- 2 Hyperparameter Tuning
- 3 Using other models like unsupervised machine learning to get the better performance and find the most important feature that cause heart disease.
- 4 Application Development.

Reference

- [1] github.com/harinda0/Heart-Disease-Prediction-ML
- [2] [geeksforgeeks.org/machine-learning/machine-learning/](https://www.geeksforgeeks.org/machine-learning/machine-learning/)
- [3] [who.int/news-room/fact-sheets/detail/cardiovascular-diseases-cvds](https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-cvds)

Thank You!

